HEC National Skill Competency Test

Data Structures & Algorithms

Complete Study Notes

All 12 Topics • With Examples & Complexity Analysis

Topic 1 • Foundations of Data Structures & Algorithms

Core concepts, ADT, memory & complexity basics

1.1 What is a Data Structure?

A **data structure** is a systematic way of organizing, managing, and storing data in a computer so that it can be accessed and modified efficiently. It defines the relationship between data elements and the operations that can be performed on them.

Why Data Structures Matter

- Efficient data access reduces time complexity of programs.
- Correct choice of DS can reduce memory usage (space complexity).
- Real-world software (OS, databases, compilers) relies on appropriate DSes.
- Enables scalable solutions — important for large datasets.

1.2 Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a mathematical model of a data type defined by its *behavior* (operations) rather than its implementation. It separates *what* a data structure does from *how* it does it.

Example — Stack ADT:

Operations: push(x), pop(), peek(), isEmpty(), size(). The ADT does not specify whether the stack is implemented using an array or a linked list — that is an implementation detail.

1.3 Classification of Data Structures

Category	Type	Examples
Primitive	Basic types	int, float, char, boolean
Non-Primitive / Linear	Sequential	Array, Stack, Queue, Linked List
Non-Primitive / Non-Linear	Hierarchical / Network	Tree, Graph, Heap, Trie
Static	Fixed size	Array
Dynamic	Grows/shrinks	Linked List, Dynamic Array

1.4 Algorithm — Properties & Characteristics

An **algorithm** is a finite, well-defined sequence of instructions that solves a problem. Every algorithm must satisfy:

- **Finiteness:** Must terminate after a finite number of steps.

- **Definiteness:** Each step must be precisely and unambiguously defined.
- **Input:** Zero or more inputs supplied externally.
- **Output:** At least one output is produced.
- **Effectiveness:** Each step must be simple enough to be carried out exactly.

1.5 Time & Space Complexity (Introduction)

Time complexity measures how the runtime of an algorithm grows with input size *n*. **Space complexity** measures the memory an algorithm uses relative to input size.

Big-O Notation — Common Classes (fastest → slowest):

Notation	Name	Example Algorithm
`O(1)`	Constant	Array access by index
`O(log n)`	Logarithmic	Binary Search
`O(n)`	Linear	Linear Search
`O(n log n)`	Linearithmic	Merge Sort, Heap Sort
`O(n²)`	Quadratic	Bubble Sort, Insertion Sort
`O(2■)`	Exponential	Recursive Fibonacci (naive)
`O(n!)`	Factorial	Brute-force Travelling Salesman

Key Rule of Thumb:

When calculating Big-O: drop constants and lower-order terms. E.g., $T(n) = 5n^2 + 3n + 7 \rightarrow O(n^2)$.

Topic 2 • Linear Data Structures

Arrays, Linked Lists, Stacks, Queues

2.1 Arrays

An **array** is a contiguous block of memory storing elements of the same type. Elements are accessed via a zero-based index in O(1) time.

Operation	Best	Average	Worst	Space
Access by index	O(1)	O(1)	O(1)	O(n)
Search (unsorted)	O(1)	O(n)	O(n)	O(1)
Insert at end	O(1)	O(1)	O(1)	–
Insert at middle	O(n)	O(n)	O(n)	–
Delete at end	O(1)	O(1)	O(1)	–

Types of Arrays:

- **1-D Array:** Linear list: int arr[5] = {1,2,3,4,5}
- **2-D Array:** Matrix: int mat[3][3]; accessed as mat[i][j]
- **Dynamic Array:** Resizes automatically (e.g., Python list, C++ vector). Amortized O(1) append.

Example — 2D Array (Matrix):

```
int matrix[3][3] = {
{1, 2, 3},
{4, 5, 6},
{7, 8, 9}
};
// Access element at row 1, col 2 → matrix[1][2] = 6 (O(1))
```

2.2 Linked List

A **linked list** is a sequence of nodes where each node contains **data** and a **pointer** to the next node. Unlike arrays, nodes are not stored contiguously in memory.

Types:

- **Singly Linked List (SLL):** Each node points to the next. Traversal is one-directional.
- **Doubly Linked List (DLL):** Each node has two pointers: next and prev. Allows bidirectional traversal.
- **Circular Linked List:** Last node points back to the head. Useful for round-robin scheduling.

```
// Singly Linked List Node (C)
struct Node {
int data;
struct Node* next;
```

```
};

// Insert at head — O(1)
void insertAtHead(struct Node** head, int val) {
struct Node* newNode = malloc(sizeof(struct Node));
newNode->data = val;
newNode->next = *head;
*head = newNode;
}
```

Operation	Best	Average	Worst	Space
Access (by index)	O(n)	O(n)	O(n)	O(n)
Search	O(1)	O(n)	O(n)	–
Insert at head	O(1)	O(1)	O(1)	O(1)
Insert at tail	O(n)	O(n)	O(n)	–
Delete at head	O(1)	O(1)	O(1)	–

2.3 Stack

A **stack** is a LIFO (Last In, First Out) linear data structure. Elements are added and removed from the same end, called the **top**.

Core Operations:

- **push(x):** Add element x to the top — O(1)
- **pop():** Remove and return the top element — O(1)
- **peek() / top():** View the top element without removing — O(1)
- **isEmpty():** Check if stack has no elements — O(1)

Applications of Stack:

- Function call management (call stack)
- Undo/Redo operations in editors
- Expression evaluation (infix → postfix)
- Balanced parentheses checking
- Depth-First Search (DFS) in graphs

```
# Python Stack using list
stack = []
stack.append(10) # push
stack.append(20)
stack.append(30)
print(stack.pop()) # → 30 (LIFO)
print(stack[-1]) # → 20 (peek)
```

Balanced Parentheses Example:

Input: "({[]})" → Push each opening bracket. When closing bracket met, pop and check match. If all match and stack empty → BALANCED.

2.4 Queue

A **queue** is a FIFO (First In, First Out) linear data structure. Elements are added at the **rear** (enqueue) and removed from the **front** (dequeue).

- **Simple Queue:** Basic FIFO. Problem: wasted space after dequeue.
- **Circular Queue:** Rear wraps around to front. Fixes space wastage. Used in OS scheduling.
- **Double-Ended Queue (Deque):** Insert and delete from both ends.
- **Priority Queue:** Element with highest priority is served first. Implemented using Heap.

```
from collections import deque
q = deque()
q.append(10) # enqueue rear
q.append(20)
q.append(30)
print(q.popleft()) # dequeue front → 10 (FIFO)
```

Applications of Queue:

- CPU scheduling (Round Robin)
- BFS graph traversal
- Printer spooling
- Network packet buffering
- Ticket booking systems

Topic 3 • Non-Linear Data Structures

Trees, Graphs, Heaps, Tries

3.1 Trees — Overview

A **tree** is a hierarchical (non-linear) data structure with a **root** node and subtrees of children. Unlike graphs, trees have no cycles. With N nodes, a tree has N-1 edges.

Key Terminology:

- **Root:** Top-most node with no parent.
- **Leaf:** Node with no children.
- **Height of tree:** Longest path from root to a leaf.
- **Depth of node:** Distance from root to that node.
- **Degree:** Number of children of a node.
- **Subtree:** A node and all its descendants.

3.2 Binary Tree

A **Binary Tree** is a tree where each node has at most 2 children: **left** and **right**.

Types of Binary Trees:

- **Full BT:** Every node has 0 or 2 children.
- **Complete BT:** All levels filled except possibly the last, which is filled left to right.
- **Perfect BT:** All internal nodes have 2 children; all leaves are at the same level.
- **Skewed BT:** Every node has only one child (left-skewed or right-skewed). Worst case for BST.
- **Balanced BT:** Height is O(log n). AVL and Red-Black trees are examples.

3.3 Binary Search Tree (BST)

A **BST** is a binary tree with the ordering property: for any node with value *k*, all values in the LEFT subtree are < k and all values in the RIGHT subtree are > k.

```
# BST Search (Python)
def search(root, key):
if root is None or root.val == key:
return root
if key < root.val:
return search(root.left, key) # go left
return search(root.right, key) # go right
# Time: O(log n) balanced, O(n) worst (skewed)
```

Operation	Best	Average	Worst	Space
Search	O(log n)	O(log n)	O(n)	O(n)

Operation	Best	Average	Worst	Space
Insert	O(log n)	O(log n)	O(n)	O(1)
Delete	O(log n)	O(log n)	O(n)	O(1)

3.4 Tree Traversals

Traversal visits every node exactly once. For binary tree with nodes 4, 2, 6, 1, 3, 5, 7 (root=4):

- **In-order (LNR):** Left → Root → Right → Result: *1 2 3 4 5 6 7 ← sorted order for BST*
- **Pre-order (NLR):** Root → Left → Right → Result: *4 2 1 3 6 5 7 ← used for tree copying*
- **Post-order (LRN):** Left → Right → Root → Result: *1 3 2 5 7 6 4 ← used for deletion*
- **Level-order (BFS):** Level by level → Result: *4 2 6 1 3 5 7 ← uses a queue*

3.5 Graph

A **graph** G = (V, E) consists of a set of **vertices** V and **edges** E connecting pairs of vertices. Unlike trees, graphs can have cycles.

Types of Graphs:

- **Directed Graph (Digraph):** Edges have direction: (u → v) ≠ (v → u).
- **Undirected Graph:** Edges are bidirectional: {u, v} = {v, u}.
- **Weighted Graph:** Each edge has a numerical weight (e.g., distance).
- **Connected Graph:** There is a path between every pair of vertices.
- **Cyclic / Acyclic:** Contains / does not contain a cycle.
- **DAG:** Directed Acyclic Graph — used for task scheduling, dependency resolution.

Graph Representations:

- **Adjacency Matrix:** 2D array adj[u][v]=1 if edge exists. Space: O(V^2). Fast edge lookup O(1).
- **Adjacency List:** Array of lists. Space: O(V+E). Better for sparse graphs.

3.6 Heap

A **heap** is a complete binary tree satisfying the heap property. Usually implemented using an array.

- **Max-Heap:** Parent ≥ children. Root is always the maximum element.
- **Min-Heap:** Parent ≤ children. Root is always the minimum element.

For a node at index i: Left child = 2i+1, Right child = 2i+2, Parent = (i-1)//2

Operation	Best	Average	Worst	Space
Insert	O(log n)	O(log n)	O(log n)	O(n)
Get Max/Min (peek)	O(1)	O(1)	O(1)	–

Operation	Best	Average	Worst	Space
Extract Max/Min	$O(\log n)$	$O(\log n)$	$O(\log n)$	–
Build Heap	$O(n)$	$O(n)$	$O(n)$	–

Topic 4 • Searching Algorithms

Linear Search, Binary Search, Interpolation, Jump, Exponential

4.1 Linear Search

Sequentially checks every element until the target is found or the list ends. Works on **unsorted** data.

```
def linear_search(arr, target):
for i in range(len(arr)):
if arr[i] == target:
return i # found at index i
return -1 # not found
# Example: arr=[5,3,8,1,9], target=8 → checks 5,3,8 → found at index 2
```

Operation	Best	Average	Worst	Space
Linear Search	O(1) (first)	O(n)	O(n)	O(1)

4.2 Binary Search

Efficiently searches a **sorted** array by repeatedly halving the search space. Compares the target with the middle element and eliminates half the array each time.

```
def binary_search(arr, target):
low, high = 0, len(arr) - 1
while low <= high:
mid = (low + high) // 2
if arr[mid] == target:
return mid
elif arr[mid] < target:
low = mid + 1 # search right half
else:
high = mid - 1 # search left half
return -1

# Example: arr=[1,3,5,7,9,11], target=7
# Step1: mid=5→ arr[2]=5 < 7, low=3
# Step2: mid=4→ arr[4]=9 > 7, high=3
# Step3: mid=3→ arr[3]=7 == 7 → return 3
```

Operation	Best	Average	Worst	Space
Binary Search	O(1)	O(log n)	O(log n)	O(1)

4.3 Jump Search

Works on **sorted arrays**. Jumps ahead by fixed steps (√n) then does linear search in the block where target may lie. Better than linear, not as fast as binary.

Optimal jump step = √n. Time: O(√n). Space: O(1).

4.4 Interpolation Search

Improved binary search for **uniformly distributed sorted data**. Uses a probe position formula:

probe = low + [(target - arr[low]) / (arr[high] - arr[low])] × (high - low)

Operation	Best	Average	Worst	Space
Interpolation	`O(1)`	`O(log log n)`	`O(n)`	`O(1)`

4.5 Exponential Search

Finds the range where target may exist by doubling the index, then applies binary search within that range. Useful for **unbounded / infinite** sorted arrays.

Time: O(log n). Space: O(1).

Searching Algorithm Comparison

- Linear Search: O(n) — works on unsorted data.
- Binary Search: O(log n) — requires sorted data, best general-purpose.
- Jump Search: O(√n) — sorted data, simpler than binary.
- Interpolation Search: O(log log n) avg — sorted & uniform distribution.
- Exponential Search: O(log n) — sorted, unbounded arrays.

Topic 5 • Sorting Algorithms

Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix

5.1 Bubble Sort

Repeatedly compares adjacent elements and swaps them if out of order. Largest element 'bubbles' to the end each pass.

```
def bubble_sort(arr):
n = len(arr)
for i in range(n-1):
swapped = False
for j in range(n-1-i):
if arr[j] > arr[j+1]:
arr[j], arr[j+1] = arr[j+1], arr[j]
swapped = True
if not swapped: break # optimization — already sorted

# Trace: [5,3,8,1] → [3,5,1,8] → [3,1,5,8] → [1,3,5,8]
```

Operation	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)

5.2 Selection Sort

Finds the minimum element in the unsorted portion and places it at the beginning. Makes exactly n-1 swaps — good when write operations are costly.

Operation	Best	Average	Worst	Space
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)

5.3 Insertion Sort

Builds the sorted array one element at a time by inserting each element into its correct position. Efficient for small or nearly sorted data.

```
def insertion_sort(arr):
for i in range(1, len(arr)):
key = arr[i]
j = i - 1
while j >= 0 and arr[j] > key:
arr[j+1] = arr[j]
j -= 1
arr[j+1] = key
# Trace: [5,3,8,1]
# i=1: key=3, [3,5,8,1]
# i=2: key=8, [3,5,8,1]
# i=3: key=1, [1,3,5,8]
```

Operation	Best	Average	Worst	Space
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

5.4 Merge Sort

Divide and Conquer: Divides array into halves, recursively sorts them, then merges the sorted halves. Guaranteed O(n log n). **Stable sort.**

```
def merge_sort(arr):
if len(arr) <= 1: return arr
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
return merge(left, right)

def merge(L, R):
result, i, j = [], 0, 0
while i < len(L) and j < len(R):
if L[i] <= R[j]: result.append(L[i]); i += 1
else: result.append(R[j]); j += 1
return result + L[i:] + R[j:]
```

Operation	Best	Average	Worst	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

5.5 Quick Sort

Divide and Conquer: Picks a **pivot**, partitions array so elements less than pivot are on left and greater are on right, then recursively sorts both partitions. **In-place, not stable.**

```
def quick_sort(arr, low, high):
if low < high:
pi = partition(arr, low, high)
quick_sort(arr, low, pi - 1)
quick_sort(arr, pi + 1, high)

def partition(arr, low, high): # Lomuto scheme
pivot = arr[high]
i = low - 1
for j in range(low, high):
if arr[j] <= pivot:
i += 1
arr[i], arr[j] = arr[j], arr[i]
arr[i+1], arr[high] = arr[high], arr[i+1]
return i + 1
```

Operation	Best	Average	Worst	Space
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Note: *Worst case $O(n^2)$ occurs when pivot is always smallest/largest (e.g., sorted array). Use random pivot or median-of-three to avoid this.*

5.6 Heap Sort

Builds a max-heap, then repeatedly extracts the maximum element placing it at the end. **In-place, not stable.** Guaranteed O(n log n).

Operation	Best	Average	Worst	Space
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

5.7 Counting Sort

Non-comparison sort. Counts occurrences of each element. Works only when elements are non-negative integers within a known range k. Time: O(n+k), Space: O(k).

5.8 Radix Sort

Sorts integers digit by digit from least significant to most significant using Counting Sort as subroutine. Time: O(d(n+k)) where d = number of digits.

Sorting Algorithm Summary

- Stable sorts (preserve order of equal elements): Bubble, Insertion, Merge, Counting, Radix.
- In-place sorts (O(1) extra space): Bubble, Selection, Insertion, Quick, Heap.
- Best average case O(n log n): Merge Sort, Quick Sort, Heap Sort.
- For nearly sorted data: Insertion Sort is best (O(n)).
- For integers in small range: Counting/Radix beats comparison sorts.

Topic 6 • Hashing

Hash Functions, Collision Resolution, Hash Tables

6.1 What is Hashing?

Hashing converts a key into an array index using a **hash function** h(k). The goal is to achieve O(1) average-case time for insert, search, and delete.

Hash Table: An array of size m where element with key k is stored at index h(k).

6.2 Hash Functions

- **Division Method:** h(k) = k mod m — m should be a prime number far from powers of 2.
- **Multiplication Method:** h(k) = ⌊m × (k × A mod 1)⌋ — A ≈ 0.6180339887 (golden ratio). Works for any m.
- **Universal Hashing:** Random hash function from a family — Prevents adversarial worst-case inputs.

Example: m=10, h(k) = k mod 10. Keys: 23, 45, 12, 33

h(23)=3, h(45)=5, h(12)=2, h(33)=3 ← collision at index 3!

6.3 Collision Resolution

A **collision** occurs when two keys hash to the same index. Strategies:

A) Separate Chaining (Open Hashing):

Each hash table slot stores a linked list of all keys that hash to that index. Easy to implement, no table size constraint. Worst case O(n) if all keys collide.

```
# Hash table with chaining (Python)
class HashTable:
def __init__(self, size=10):
self.table = [[] for _ in range(size)]
def insert(self, key):
idx = key % len(self.table)
self.table[idx].append(key)
def search(self, key):
idx = key % len(self.table)
return key in self.table[idx]
```

B) Open Addressing (Closed Hashing):

All elements stored in the hash table itself. If collision occurs, probe for next slot.

- **Linear Probing:** h(k, i) = (h(k) + i) mod m — clusters form ('primary clustering')
- **Quadratic Probing:** h(k, i) = (h(k) + i²) mod m — reduces primary clustering
- **Double Hashing:** h(k, i) = (h1(k) + i × h2(k)) mod m — best distribution

6.4 Load Factor & Rehashing

Load factor $\alpha = n / m$ (n = elements, m = table size). When α exceeds a threshold (typically 0.7), rehashing doubles the table size and reinserts all elements — amortized O(1) operations.

Operation	Best	Average	Worst	Space
Search	`O(1)`	`O(1)`	`O(n)`	`O(n)`
Insert	`O(1)`	`O(1)`	`O(n)`	–
Delete	`O(1)`	`O(1)`	`O(n)`	–

6.5 Applications of Hashing

- Hash Maps / Dictionaries (Python dict, Java HashMap)
- Database indexing
- Caching (memoization, LRU cache)
- Cryptographic hash functions (SHA-256, MD5)
- Detecting duplicate files / plagiarism
- Set membership testing (Bloom filters)

Topic 7 • Tree Algorithms

AVL, Red-Black, B-Trees, Segment Trees, Fenwick Trees

7.1 AVL Tree (Self-Balancing BST)

An **AVL Tree** is a BST that automatically keeps itself balanced. The **Balance Factor** of every node must be -1, 0, or +1. BF = Height(left subtree) - Height(right subtree).

Rotations to restore balance:

- **Right Rotation (LL case):** Left-Left imbalance: new node inserted in left subtree of left child.
- **Left Rotation (RR case):** Right-Right imbalance: new node inserted in right subtree of right child.
- **Left-Right Rotation (LR):** Left-Right imbalance: left rotate child, then right rotate root.
- **Right-Left Rotation (RL):** Right-Left imbalance: right rotate child, then left rotate root.

Operation	Best	Average	Worst	Space
Search	`O(log n)`	`O(log n)`	`O(log n)`	`O(n)`
Insert	`O(log n)`	`O(log n)`	`O(log n)`	–
Delete	`O(log n)`	`O(log n)`	`O(log n)`	–

7.2 Red-Black Tree

A **Red-Black Tree** is a self-balancing BST where each node has a color (red/black). Used in many real-world implementations (C++ STL map, Java TreeMap).

Red-Black Properties:

- Every node is either RED or BLACK.
- The root is always BLACK.
- Every leaf (NIL/null) is BLACK.
- If a node is RED, both children are BLACK (no two consecutive reds).
- All paths from any node to its descendant NIL leaves have the same number of BLACK nodes.

Height: At most 2 log(n+1). Operations: O(log n).

7.3 B-Tree

A **B-Tree of order m** is a self-balancing search tree where each node can have multiple keys and children. Designed for **disk-based storage** (databases, file systems).

Properties (order m):

- Each internal node has at most m children and m-1 keys.
- Each internal node (except root) has at least ⌈m/2⌉ children.

- Root has at least 2 children if it is not a leaf.
- All leaves are at the same depth.
- Keys within a node are sorted.

B+ Tree: All data stored in leaves; internal nodes only store keys for routing. Leaves linked for range queries. Used in databases (MySQL InnoDB).

7.4 Segment Tree

A **Segment Tree** is a binary tree used for range queries (e.g., range sum, range min/max) and point updates on an array. Each node stores aggregate info about a segment.

Build: O(n). Query: O(log n). Update: O(log n). Space: O(n).

```
# Segment Tree — Range Sum Query
def build(arr, tree, node, start, end):
if start == end:
tree[node] = arr[start]
else:
mid = (start + end) // 2
build(arr, tree, 2*node, start, mid)
build(arr, tree, 2*node+1, mid+1, end)
tree[node] = tree[2*node] + tree[2*node+1]
```

7.5 Fenwick Tree (Binary Indexed Tree)

A **Fenwick Tree (BIT)** is an array-based structure for efficient prefix sum queries and point updates. Simpler to implement than Segment Tree.

Query (prefix sum): O(log n). Update: O(log n). Space: O(n).

```
def update(bit, i, delta, n):
while i <= n:
bit[i] += delta
i += i & (-i) # move to parent

def query(bit, i):
s = 0
while i > 0:
s += bit[i]
i -= i & (-i) # move to responsible ancestor
return s
```

Topic 8 • Graph Algorithms

BFS, DFS, Dijkstra, Bellman-Ford, Floyd-Warshall, MST

8.1 Breadth-First Search (BFS)

Explores graph level by level using a **queue**. Visits all neighbours of a vertex before moving to next level. Finds **shortest path in unweighted graphs**.

```
from collections import deque
def bfs(graph, start):
visited = set([start])
queue = deque([start])
order = []
while queue:
v = queue.popleft()
order.append(v)
for neighbour in graph[v]:
if neighbour not in visited:
visited.add(neighbour)
queue.append(neighbour)
return order
# Time: O(V+E), Space: O(V)
```

Applications: Shortest path (unweighted), Level-order traversal, Finding connected components, Bipartite graph check.

8.2 Depth-First Search (DFS)

Explores as deep as possible using a **stack** (or recursion). Backtracks when a dead-end is reached.

```
def dfs(graph, v, visited=None):
if visited is None: visited = set()
visited.add(v)
print(v, end=' ')
for neighbour in graph[v]:
if neighbour not in visited:
dfs(graph, neighbour, visited)
# Time: O(V+E), Space: O(V)
```

Applications: Topological sort, Cycle detection, Path finding, Strongly connected components (Tarjan's / Kosaraju's).

8.3 Dijkstra's Algorithm (Single Source Shortest Path)

Finds shortest paths from source to all vertices in a **weighted graph with non-negative edges**. Uses a **min-heap (priority queue)**.

```
import heapq
def dijkstra(graph, src, V):
dist = [float('inf')] * V
dist[src] = 0
```

```
pq = [(0, src)] # (distance, vertex)
while pq:
d, u = heapq.heappop(pq)
if d > dist[u]: continue
for v, w in graph[u]:
if dist[u] + w < dist[v]:
dist[v] = dist[u] + w
heapq.heappush(pq, (dist[v], v))
return dist
# Time: O((V+E) log V) with binary heap
```

Limitation: *Does NOT work with negative weight edges.*

8.4 Bellman-Ford Algorithm

Handles **negative weight edges**. Relaxes all edges V-1 times. Can detect **negative weight cycles**. Slower than Dijkstra.

Time: O(V × E). Space: O(V).

Negative cycle detected if any edge can still be relaxed after V-1 iterations.

8.5 Floyd-Warshall Algorithm (All-Pairs Shortest Path)

Finds shortest paths between **all pairs** of vertices. Uses dynamic programming. Works with negative edges but not negative cycles.

```
def floyd_warshall(dist, V):
for k in range(V): # intermediate vertex
for i in range(V):
for j in range(V):
dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
# Time: O(V³), Space: O(V²)
```

8.6 Minimum Spanning Tree (MST)

A **spanning tree** of a connected graph connects all vertices with minimum total edge weight and no cycles. Two famous algorithms:

A) Kruskal's Algorithm:

- Sort all edges by weight.
- Add the smallest edge that does not form a cycle (use Union-Find to detect cycles).
- Repeat until V-1 edges are added.
- Time: O(E log E). Best for sparse graphs.

B) Prim's Algorithm:

- Start from any vertex. Maintain a set of vertices included in MST.
- Greedily pick the minimum weight edge connecting MST to a non-MST vertex.
- Use priority queue for efficiency.
- Time: O((V+E) log V). Better for dense graphs.

8.7 Topological Sort

Linear ordering of vertices in a **DAG** such that for every directed edge (u → v), u comes before v. Used for task scheduling, dependency resolution.

Methods: DFS-based (post-order reversal) or Kahn's algorithm (BFS + in-degree). Time: O(V+E).

Topic 9 • Algorithm Design Techniques

Divide & Conquer, DP, Greedy, Backtracking, Branch & Bound

9.1 Divide and Conquer

Breaks a problem into smaller sub-problems, solves them independently, then combines results. Sub-problems must be **independent**.

Steps: Divide → Conquer (recurse) → Combine.

Examples of Divide & Conquer

- Merge Sort: Split array, sort halves, merge. O(n log n).
- Quick Sort: Partition around pivot, sort partitions.
- Binary Search: Eliminate half the search space each step.
- Strassen's Matrix Multiplication: O(n^2.81) vs O(n³).
- Closest Pair of Points: O(n log n) geometric algorithm.

Master Theorem for Recurrences:

T(n) = aT(n/b) + f(n) where a ≥ 1, b > 1.

- **Case 1:** If f(n) = O(n^(log_b a - ε)) → T(n) = Θ(n^log_b a)
- **Case 2:** If f(n) = Θ(n^log_b a) → T(n) = Θ(n^log_b a × log n)
- **Case 3:** If f(n) = Ω(n^(log_b a + ε)) → T(n) = Θ(f(n))

Example: Merge Sort T(n) = 2T(n/2) + O(n): a=2, b=2, f(n)=n = n^log■2 → Case 2 → O(n log n)

9.2 Dynamic Programming (DP)

Solves problems by breaking them into **overlapping sub-problems** and storing results to avoid recomputation (**memoization**). Requires **optimal substructure**.

Two approaches:

- **Top-Down (Memoization):** Recursive + cache results in a table.
- **Bottom-Up (Tabulation):** Iteratively fill table from base cases up.

Example 1: Fibonacci

```
# Naive recursive: O(2■)
# DP memoization: O(n)
def fib(n, memo={}):
if n <= 1: return n
if n not in memo:
memo[n] = fib(n-1, memo) + fib(n-2, memo)
return memo[n]
```

Example 2: 0/1 Knapsack Problem

Given n items with weights w[] and values v[], and knapsack capacity W: Maximize total value without exceeding capacity. Each item either included (1) or excluded (0).

```
def knapsack(W, wt, val, n):
dp = [[0]*(W+1) for _ in range(n+1)]
for i in range(1, n+1):
for w in range(W+1):
if wt[i-1] <= w:
dp[i][w] = max(dp[i-1][w],
val[i-1] + dp[i-1][w - wt[i-1]])
else:
dp[i][w] = dp[i-1][w]
return dp[n][W]
# Time: O(nW), Space: O(nW)
```

Example 3: Longest Common Subsequence (LCS)

Find the longest subsequence common to two strings. LCS("ABCBDAB", "BDCAB") = 4 ("BCAB" or "BDAB")

```
def lcs(X, Y):
m, n = len(X), len(Y)
dp = [[0]*(n+1) for _ in range(m+1)]
for i in range(1, m+1):
for j in range(1, n+1):
if X[i-1] == Y[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
return dp[m][n] # Time: O(mn), Space: O(mn)
```

9.3 Greedy Algorithm

Makes the locally optimal choice at each step hoping to reach a global optimum. Fast but does not always produce the globally optimal solution. Works when problem has **greedy choice property** and **optimal substructure**.

Famous Greedy Problems

- Activity Selection: Select max non-overlapping activities — sort by finish time.
- Fractional Knapsack: Take fractions of items — sort by value/weight ratio.
- Huffman Coding: Optimal prefix codes for data compression.
- Prim's & Kruskal's MST: Greedily pick minimum edges.
- Dijkstra's SSSP: Greedily relax the closest unvisited vertex.

9.4 Backtracking

Systematically explores all possible solutions by trying candidates and abandoning (**backtracking**) those that fail constraints. Better than brute force but still exponential.

Example: N-Queens Problem

Place N queens on N×N chessboard so no two queens attack each other. Place queen in column 0, try each row, check safety, recurse for column 1, backtrack if stuck.

```
def solve_nqueens(board, col, n):
if col == n: return True
for row in range(n):
if is_safe(board, row, col, n):
board[row][col] = 1
if solve_nqueens(board, col+1, n): return True
board[row][col] = 0 # backtrack
return False
```

Other backtracking examples: Sudoku solver, Rat in a Maze, Graph coloring, Subset Sum, Permutations.

9.5 Branch and Bound

Used for optimization problems. Like backtracking but uses a **bound function** to prune branches that cannot lead to a better solution than the current best. More systematic than backtracking.

Examples: 0/1 Knapsack (optimal), Travelling Salesman Problem (TSP), Job Assignment.

Topic 10 • Advanced Data Structures

Trie, Disjoint Set, Sparse Table, Skip List, Bloom Filter

10.1 Trie (Prefix Tree)

A **Trie** is an n-ary tree used to store strings. Each node represents a character. Path from root to a node spells a string prefix. Very efficient for **prefix-based searches**.

Example: Insert 'cat', 'car', 'card', 'care' into a trie:

Root → c → a → t (end), r (end) → d (end), e (end)

```
class TrieNode:
def __init__(self):
self.children = {}
self.is_end = False

class Trie:
def __init__(self): self.root = TrieNode()
def insert(self, word):
node = self.root
for ch in word:
if ch not in node.children:
node.children[ch] = TrieNode()
node = node.children[ch]
node.is_end = True
def search(self, word):
node = self.root
for ch in word:
if ch not in node.children: return False
node = node.children[ch]
return node.is_end
```

Operation	Best	Average	Worst	Space
Insert	O(L)	O(L)	O(L)	O(ALPHABET × N × L)
Search	O(L)	O(L)	O(L)	–
Prefix Search	O(L)	O(L)	O(L)	–

L = length of word, N = number of words.

Applications: Autocomplete, Spell checker, IP routing, DNA sequencing.

10.2 Disjoint Set Union (Union-Find)

A data structure that tracks a set of elements partitioned into **disjoint subsets**. Supports two operations efficiently:

- **Find(x):** Returns the representative (root) of the set containing x.

• **Union(x, y):** Merges the sets containing x and y.

```
class DSU:
def __init__(self, n):
self.parent = list(range(n))
self.rank = [0] * n
def find(self, x): # with path compression
if self.parent[x] != x:
self.parent[x] = self.find(self.parent[x])
return self.parent[x]
def union(self, x, y): # union by rank
px, py = self.find(x), self.find(y)
if px == py: return
if self.rank[px] < self.rank[py]: px, py = py, px
self.parent[py] = px
if self.rank[px] == self.rank[py]: self.rank[px] += 1
# Amortized O(α(n)) per operation — nearly O(1)!
```

Applications: Kruskal's MST, Cycle detection, Network connectivity.

10.3 Sparse Table

A data structure for answering **Range Minimum/Maximum Queries (RMQ)** in O(1) time after O(n log n) preprocessing. Works on **static arrays** (no updates).

Key idea: Precompute min/max for all intervals of length 2^j. For any query [l, r]: use two overlapping precomputed intervals.

Operation	Best	Average	Worst	Space
Build	—	—	`O(n log n)`	`O(n log n)`
RMQ Query	—	—	`O(1)`	—

10.4 Skip List

A probabilistic data structure that allows fast search, insert, and delete in a sorted list. Multiple layered linked lists with express lanes that skip over many elements.

Operation	Best	Average	Worst	Space
Search	`O(log n)`	`O(log n)`	`O(n)`	`O(n log n)`
Insert	`O(log n)`	`O(log n)`	`O(n)`	—
Delete	`O(log n)`	`O(log n)`	`O(n)`	—

10.5 Bloom Filter

A space-efficient probabilistic data structure for **set membership testing**. May produce **false positives** (says element is in set when it's not) but **never false negatives**.

Uses k hash functions and a bit array. Used in: databases (avoid disk lookups), web caches, spam filters, password checking.

Topic 11 • String Algorithms

Pattern Matching, KMP, Rabin-Karp, Z-Algorithm, Suffix Arrays

11.1 Naive Pattern Matching

For text T of length n and pattern P of length m: Check every position in T for a match. Time: O(nm). Works but slow for large inputs.

```
def naive_search(text, pattern):
n, m = len(text), len(pattern)
for i in range(n - m + 1):
if text[i:i+m] == pattern:
print(f'Pattern found at index {i}')
```

11.2 KMP (Knuth-Morris-Pratt) Algorithm

Preprocessing: Compute the **Failure Function (LPS array)** — Longest Proper Prefix which is also a Suffix. On mismatch, don't restart from beginning; use LPS to skip characters.

```
def compute_lps(pattern):
m = len(pattern)
lps = [0] * m
length, i = 0, 1
while i < m:
if pattern[i] == pattern[length]:
length += 1; lps[i] = length; i += 1
elif length != 0:
length = lps[length - 1]
else:
lps[i] = 0; i += 1
return lps

# KMP Search: Time O(n+m), Space O(m)
# Example: pattern='ABAB', LPS=[0,0,1,2]
# On mismatch at pos 2 with length=2: jump to lps[1]=0
```

Operation	Best	Average	Worst	Space
KMP	O(n+m)	O(n+m)	O(n+m)	O(m)

11.3 Rabin-Karp Algorithm

Uses **rolling hash** to compare hash of pattern with hash of text windows. Only does character-by-character comparison on hash matches (to handle collisions).

Average: O(n+m). Worst (many hash collisions): O(nm). Excellent for **multiple pattern search**.

```
# Rolling hash: remove leftmost char, add rightmost char — O(1) update
# hash = (d * hash + char) mod q
# where d = alphabet size, q = prime number
```

11.4 Z-Algorithm

Computes Z-array where Z[i] = length of the longest substring starting at i that is also a prefix of the string. Useful for pattern matching in O(n+m).

Concatenate P + '$' + T, compute Z-array. Positions where Z[i] = m are pattern occurrences.

11.5 Suffix Array & Suffix Tree

A **Suffix Array** is a sorted array of all suffixes of a string. Enables many string operations in O(n log n) or O(n) time.

With **LCP array** (Longest Common Prefix), enables: pattern search in O(m log n), longest repeated substring, number of distinct substrings.

A **Suffix Tree** is a compressed trie of all suffixes. Build in O(n) using Ukkonen's algorithm. More powerful but complex.

11.6 String DP Problems

- **Edit Distance (Levenshtein):** Min operations (insert, delete, replace) to convert string A to B. O(mn) DP.
- **Longest Palindromic Subsequence:** LCS of string with its reverse.
- **Longest Palindromic Substring:** Expand around center O(n²) or Manacher's O(n).

String Algorithm Comparison

- Naive: O(nm) — simple, no preprocessing.
- KMP: O(n+m) — failure function, single pattern.
- Rabin-Karp: O(n+m) avg — rolling hash, multiple patterns.
- Z-Algorithm: O(n+m) — Z-array, single pattern.
- Boyer-Moore: O(n/m) best — bad character + good suffix heuristics.

Topic 12 • Complexity & Optimization

P vs NP, Amortized Analysis, Space-Time Tradeoffs, NP-Hard Problems

12.1 Asymptotic Notations

- **O(f(n)) — Big-O (Upper Bound):** Algorithm runs in AT MOST f(n) steps — worst case.
- **Ω(f(n)) — Omega (Lower Bound):** Algorithm runs in AT LEAST f(n) steps — best case.
- **Θ(f(n)) — Theta (Tight Bound):** Algorithm runs in EXACTLY f(n) steps — average case.
- **o(f(n)) — Little-o:** Strict upper bound (not tight).
- **ω(f(n)) — Little-omega:** Strict lower bound (not tight).

12.2 Amortized Analysis

Amortized analysis gives the **average cost per operation** over a sequence of operations, even if some individual operations are expensive. Methods:

- **Aggregate Method:** Total cost of n operations / n.
- **Accounting Method:** Assign credits to operations; expensive ops use stored credits.
- **Potential Method:** Define a potential function; amortized cost = actual + $\Delta\Phi$.

Example: Dynamic array (e.g., Python list) doubling. Individual append can be O(n) when resizing, but amortized cost per append is O(1) because doubling halves the frequency of resizes.

12.3 Complexity Classes

- **P:** Problems solvable in polynomial time O(n^k). E.g., Sorting, BFS, DFS, Dijkstra.
- **NP:** Non-deterministic Polynomial: solutions verifiable in polynomial time. P $\subseteq$ NP.
- **NP-Complete:** Hardest problems in NP. Every NP problem reduces to it. E.g., SAT, 3-SAT.
- **NP-Hard:** At least as hard as NP-Complete; may not be in NP. E.g., TSP optimization.
- **Co-NP:** Complement problems of NP — negatives verifiable in polynomial time.

P vs NP Question: The greatest unsolved problem in computer science. Is P = NP? If yes, every problem whose solution can be quickly verified can also be quickly solved. Most experts believe P ≠ NP.

12.4 Famous NP-Complete / NP-Hard Problems

Problem	Description	Best Known
Travelling Salesman (TSP)	Min cost tour visiting all cities once	O(n² × 2■) DP
0/1 Knapsack	Max value within weight limit	O(nW) pseudo-poly DP
Graph Coloring	Min colors so no adjacent vertices same color	O(2■ × n) DP
Subset Sum	Does any subset sum to target T?	O(nT) pseudo-poly

Vertex Cover	Min vertex set covering all edges	Approx: 2-approx
Hamiltonian Path	Path visiting all vertices exactly once	Exp. time
Boolean SAT	Is CNF formula satisfiable?	Exp. worst case

12.5 Optimization Strategies

- **Memoization / Caching:** Store expensive results; avoid recomputation. DP's top-down approach.
- **Space-Time Tradeoff:** Use extra memory to speed up computation (e.g., hash table for O(1) lookup).
- **Approximation Algorithms:** Find near-optimal solution in polynomial time for NP-Hard problems. E.g., 2-approximation for Vertex Cover; 1.5-approx for TSP (Christofides).
- **Randomized Algorithms:** Use randomness for expected fast runtime. E.g., Randomized Quick Sort, Hashing.
- **Parallel Algorithms:** Divide work across processors. E.g., Parallel Merge Sort.
- **Pruning:** Eliminate branches early in search (backtracking, branch & bound).

12.6 Recurrence Relations — Quick Reference

Recurrence	Algorithm	Result
`T(n) = T(n/2) + O(1)`	Binary Search	O(log n)
`T(n) = 2T(n/2) + O(n)`	Merge Sort	O(n log n)
`T(n) = T(n-1) + O(1)`	Factorial	O(n)
`T(n) = T(n-1) + O(n)`	Selection Sort	O(n²)
`T(n) = 2T(n-1) + O(1)`	Tower of Hanoi	O(2■)
`T(n) = 8T(n/2) + O(n²)`	Strassen's Matrix Multiply	O(n^2.81)

12.7 Space Complexity Analysis

- **Iterative algorithms (loops):** O(1) — no extra stack space
- **Recursive algorithms:** O(depth) — call stack
- **Merge Sort:** O(n) — auxiliary array for merging
- **Quick Sort:** O(log n) avg — recursive call stack
- **BFS:** O(V) — queue can hold all vertices in worst case
- **DFS:** O(V) — recursion stack depth
- **DP (2D table):** O(mn) — can often optimize to O(n) using rolling array

---

QUICK REVISION: Key Complexities to Remember

• Array access: O(1) • Linked List search: O(n) • BST ops: O(log n) avg, O(n) worst
• Hash Table: O(1) avg, O(n) worst • Heap insert/extract: O(log n) • Trie ops: O(L)
• Binary Search: O(log n) • Merge/Quick/Heap Sort: O(n log n) • Bubble/Insert/Select: $O(n^2)$
• BFS/DFS: O(V+E) • Dijkstra: O((V+E) log V) • Floyd-Warshall: $O(V^3)$
• KMP: O(n+m) • Segment Tree: O(log n) query • Union-Find: $O(\alpha(n)) \approx O(1)$